

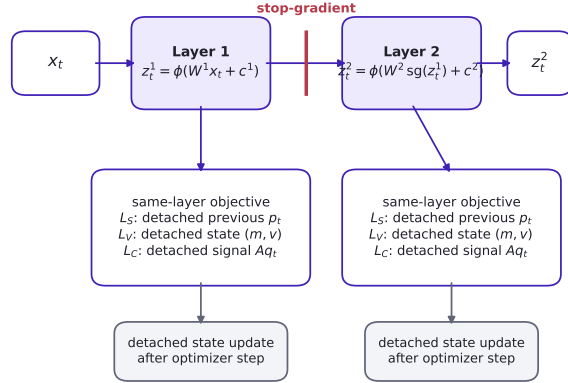
Temporal Regularized Learning: Deep Local Soft-SFA from Temporal Streams

Davide R. Wiest

Axym Labs, Darmstadt, Germany

A Local credit assignment

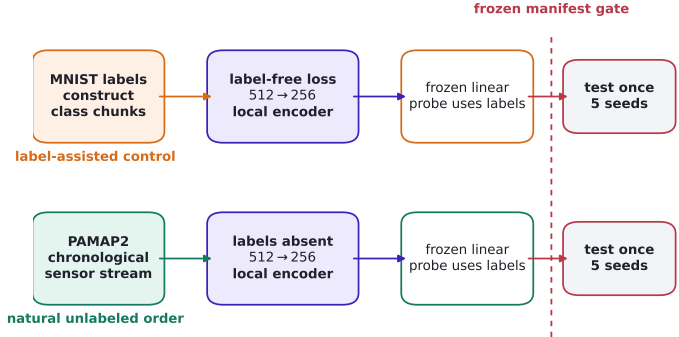
Every loss updates one nonlinear layer; graph cuts are explicit.



Streaming state per width D : $D^2 + 4D + 1$ elements; no temporal graph is retained.

B Two regimes, one guarded readout

The source of temporal adjacency determines what the experiment can claim.



Selection: validation only (seeds 101/202/303) • Confirmation: untouched test (1001-1005)

Figure 1: A: TEREL cuts gradient paths across depth and time while each layer receives temporal, variance, and lateral signals; detached state is updated after the optimizer step. **B:** label-ordered MNIST is a label-assisted control, whereas chronological PAMAP2 with labels excluded from encoder batches tests natural self-supervised order. Both use a frozen linear probe and a manifest-locked test gate.

Abstract

Temporal coherence is an attractive learning signal when credit assignment must remain within a layer, but slowness alone admits a collapsed solution. We introduce Temporal Regularized Learning (TEREL), a deep local learning rule that combines temporal coherence with gated variance expansion and a same-layer decorrelation signal. The objective is best understood as a soft neural form of Slow Feature Analysis (SFA), viewed through the variance–invariance–covariance regularization of modern self-supervised learning. Each layer is optimized independently: lower-layer activations, population statistics, temporal references, and lateral inputs are detached. We derive the exact detached and undetached gradients, specify state-update order, and show that streaming execution retains no temporal computation graph. The dynamic state is bounded in stream length but includes a dense lateral operator and is therefore quadratic in layer width. We evaluate the corrected implementation under an untouched-test protocol with matched probes and five seeds. Label-ordered MNIST serves as a controlled, explicitly label-assisted setting; PAMAP2 provides natural chronological pretraining whose activity labels are unavailable to the encoder. These experiments separate useful local representation learning from random features, supervision through ordering, and the contribution of natural temporal order.

Temporal Regularized Learning

Code: <https://github.com/Axym-Labs/TeReL>

Project page: <https://axym.org/work/terel>

Correspondence: dw@axym.org

1 Introduction

Many observations change faster than the latent causes that generate them. Object identity persists while viewpoint and illumination vary; a person’s activity persists across noisy wearable-sensor readings; and the state of a dynamical system generally evolves more slowly than its measurement noise. Temporal proximity can therefore replace hand-designed paired views as a learning signal. The difficulty is that making adjacent representations equal also makes a constant representation optimal.

Classical Slow Feature Analysis (SFA) resolves this ambiguity by minimizing temporal derivatives subject to zero-mean, unit-variance, and decorrelation constraints [1]. Modern regularized self-supervised objectives arrive at a closely related structure: an invariance term is paired with variance and covariance regularizers to prevent collapse [2, 3]. The shared structure is not itself new. Our question is instead an implementation question: can this soft-SFA objective train each nonlinear layer without gradients from later layers or through temporal history?

TEREL answers with three same-layer signals. A temporal term attracts the current activation to a detached preceding activation. A neuron-local term expands activity while its detached running variance lies below a target. A dense same-layer operator tracks off-diagonal second moments and supplies a detached approximation to the covariance-penalty gradient. Detaching the input to every subsequent layer makes the parameter update spatially local in the credit-assignment sense. Detaching the temporal reference makes the rule causal and bounded-history. Minibatches may still be used to execute the same rule efficiently; batching is not part of the locality claim.

This distinction matters because three claims are easily conflated. First, local credit assignment does not imply sparse communication: the lateral operator communicates within a layer. Second, bounded history does not imply constant state in width: a dense width- D lateral operator costs D^2 . Third, an objective without label terms is not self-supervised if labels were used to construct its temporal order. We therefore use label-ordered MNIST only as a controlled study and base the self-supervised temporal-order claim on chronological wearable streams whose labels are structurally absent from encoder batches.

The paper makes four contributions:

1. We formulate TEREL as a deep, layer-local soft-SFA rule and locate its contribution relative to SFA, Incremental SFA, Bio-SFA, VICReg, and blockwise self-supervised learning.
2. We give the implemented objective, exact activation and weight gradients for detached and undetached time, the detached state updates, and a quantitative error decomposition for the shifted lateral heuristic.
3. We demonstrate genuine batch-size-one execution and report parameters, optimizer state, dynamic state, peak memory, and throughput separately.
4. We provide corrected, leakage-free experiments with matched architectures and last-layer probes, bounded validation selection, five confirmatory seeds, raw results, uncertainty, and a held-out test gate.

2 Temporal Regularized Learning

2.1 From constrained SFA to a soft local objective

Let x_t be a temporal stream and let layer ℓ produce

$$a_t^\ell = W^\ell z_t^{\ell-1} + c^\ell, \quad z_t^\ell = \phi(a_t^\ell). \quad (1)$$

SFA searches for features that change slowly while excluding constant and duplicated solutions. In one common statement it minimizes $\mathbb{E}\|z_t - z_{t-1}\|_2^2$ subject to zero mean, unit variance, and decorrelated outputs [1]. TEREL replaces hard population constraints by detached running state and local soft terms. This is a particular neural parameterization and optimizer, not a new discovery of the SFA objective.

For one layer, consider B activations $z_t \in \mathbb{R}^D$. A Boolean boundary b_t marks the start of a subject, recording, or constructed chunk. Let $I_t = 1 - b_t$, additionally set to zero when no stored predecessor exists, and let $N_I = \sum_t I_t$. The state available before the step is a detached mean m , variance v , and lateral operator A . The canonical local loss is

$$L_S = \frac{1}{N_I D} \sum_{t: I_t=1} \|z_t - p_t\|_2^2, \quad (2)$$

$$L_V = -\frac{1}{BD} \sum_{t=1}^B \sum_{j=1}^D [\gamma - v_j]_+ (z_{tj} - m_j)^2, \quad (3)$$

$$L_C = \frac{1}{BD} \sum_{t=1}^B (z_t - m)^\top A q_t, \quad (4)$$

$$L = \lambda_S L_S + \lambda_V L_V + \lambda_C L_C. \quad (5)$$

If $N_I = 0$, L_S and its gradient are defined as zero. For TEREL-local, $p_t = \text{sg}(z_{t-1})$ (or the detached last activation from the preceding call) and $q_t = \text{sg}(z_t - m)$. Boundary flags invalidate temporal pairs.

The minus sign in Eq. (3) is essential. With the gate $g_j = [\gamma - v_j]_+$ treated as state, minimizing L_V pushes a currently active unit away from its mean until its running variance reaches γ . The term is a local gradient surrogate for a variance constraint; it is not the usual nonnegative VICReg scalar variance penalty. Coherence alone drove the original ReLU validation round into an absorbing zero state. The confirmatory architecture therefore uses LeakyReLU with fixed negative slope 0.01, and the population-statistics rate remains part of the algorithm rather than an inconsequential logging choice.

2.2 Lateral decorrelation as a gradient proxy

Define the detached off-diagonal second-moment operator

$$C = \text{offdiag} \left(\frac{1}{B} \sum_t (z_t - m)(z_t - m)^\top \right). \quad (6)$$

The direct control used in our ablations minimizes $L_{\text{direct}} = \|C\|_F^2 / D$. With m detached, its activation gradient is $4C(z_t - m) / (BD)$. TEREL instead tracks C with A and uses Eq. (4); because q_t and A are detached, the encoder receives $Aq_t / (BD)$. If $A \approx C$ and $q_t = z_t - m$, this matches the direct gradient direction up to a positive factor four. It is a proxy for the gradient, not an equality between scalar objectives.

The shifted heuristic uses the preceding centered activation as q_t . Let $u_t = z_t - m$. Its error relative to the direct signal obeys

$$\|Aq_t - Cu_t\|_2 \leq \|A - C\|_2 \|u_t\|_2 + \|A\|_2 \|q_t - u_t\|_2. \quad (7)$$

The two terms expose separate failure modes: an inaccurate lateral estimate and a representation that changes too quickly for a one-step shift. The bound does not guarantee a useful shifted signal, so we report this variant only as an ablation.

2.3 Exact local gradients

Write $g = [\gamma - v]_+$. For TEREL-local the activation gradient is

$$e_t = \frac{2\lambda_S I_t}{N_I D} (z_t - p_t) - \frac{2\lambda_V}{BD} g \odot (z_t - m) + \frac{\lambda_C}{BD} A q_t. \quad (8)$$

The input $z_t^{\ell-1}$ to each learned layer is detached. Consequently,

$$\frac{\partial L^\ell}{\partial W^\ell} = \sum_t (e_t \odot \phi'(a_t^\ell)) (z_t^{\ell-1})^\top, \quad \frac{\partial L^\ell}{\partial c^\ell} = \sum_t e_t \odot \phi'(a_t^\ell). \quad (9)$$

This is a presynaptic factor times an output-side teaching factor. Autograd computes ϕ' exactly; our derivation does not assume a linear activation. No later-layer error enters [Eq. \(9\)](#).

For comparison, TEREL-batch leaves adjacent $p_t = z_{t-1}$ inside the graph. Its temporal contribution is

$$\frac{2\lambda_S}{N_I D} [I_t(z_t - z_{t-1}) - I_{t+1}(z_{t+1} - z_t)], \quad (10)$$

with missing and boundary-crossing terms omitted. The second term depends on a future activation. We therefore describe this variant as layer-local but not temporally online.

2.4 State update and execution order

After the encoder optimizer step, state is updated under `no_grad` from the activations computed for that step:

$$m^+ = \rho_m m + (1 - \rho_m) \bar{z}, \quad (11)$$

$$v^+ = \rho_m v + (1 - \rho_m) \frac{1}{B} \sum_t (z_t - m)^2, \quad (12)$$

$$A^+ = \rho_A A + (1 - \rho_A) \text{offdiag} \left(\frac{1}{B} \sum_t (z_t - m)(z_t - m)^\top \right). \quad (13)$$

The old mean m is used in [Eqs. \(12\)](#) and [\(13\)](#). The final activation and its centered value are stored as detached temporal state. No activation graph survives the call.

Algorithm 1 One TEREL-local step for a deep encoder.

Require: input batch/stream segment X , boundary flags b , layer states $\{m^\ell, v^\ell, A^\ell, p^\ell\}$

- 1: $h \leftarrow X$
 - 2: **for** layer $\ell = 1, \dots, L$ **do**
 - 3: $z^\ell \leftarrow \phi(W^\ell \text{sg}(h) + c^\ell)$
 - 4: construct detached predecessors p_t^ℓ and valid pairs from b
 - 5: compute L^ℓ by [Eq. \(5\)](#)
 - 6: $h \leftarrow \text{sg}(z^\ell)$
 - 7: **end for**
 - 8: backpropagate $\sum_\ell L^\ell$ only within each layer; step optimizer
 - 9: **for** layer $\ell = 1, \dots, L$ **do**
 - 10: update detached state by [Eqs. \(11\)](#) to [\(13\)](#)
 - 11: **end for**
-

Table 1: Variants used in the experiments. “Time graph” and “depth graph” indicate whether auto-grad crosses those boundaries.

Method	Temporal reference	Decorrelation	Graph scope
TEREL-local	detached previous	tracked A , current q_t	same layer/current step
TEREL-batch	live previous in batch	tracked A , current q_t	same layer/batch time
TEREL-shift	detached previous	tracked A , previous q_t	same layer/current step
Direct control	detached previous	exact in-batch $\ C\ _F^2$	same layer/batch statistics
Local SupCon	label positives	supervised contrastive	same layer/current batch

2.5 Variants and claim boundary

Our primary claim is local credit assignment: parameters receive no gradient from another layer or a preceding step. The rule is online in the bounded-history sense when executed at batch size one. We do not claim biological plausibility, sparse within-layer communication, lower energy, or lower memory than every baseline.

2.6 State and memory cost

For layer width D , dynamic TEREL state contains mean D , variance D , lateral operator D^2 , previous activation D , previous centered activation D , and one validity flag: $D^2 + 4D + 1$ elements. This is constant in stream length and quadratic in width. Encoder parameters, optimizer moments, batch activations, and temporary matrix products are separate costs. In particular, AdamW adds approximately two parameter-sized moment buffers after its first step. The lateral state is discarded at inference, but its training cost is real. Batch-size-one execution removes batch growth and temporal graphs; it does not remove the dense lateral matrix.

3 Relation to prior work

SFA and incremental SFA. SFA is the objective-level ancestor of TEREL: temporal slowness, variance, and decorrelation are its defining ingredients [1]. Batch linear SFA whitens observations and solves a minor-component problem for their derivatives. IncSFA replaces those batch operations with covariance-free incremental PCA and minor-component updates, supports episodic boundaries, and has Hebbian/anti-Hebbian update forms [4]. It is therefore a stronger online precedent than a comparison to batch SFA alone. Our empirical baseline is a provenance-documented Python 3 port of the authors’ published linear algorithm, validated against batch SFA on a synthetic stream. TEREL differs by applying a soft objective directly to each nonlinear learned layer, at the cost of a dense tracked lateral operator.

Bio-SFA and trace learning. Bio-SFA derives an online neural network for the SFA problem with local synaptic updates [5]. This prevents us from presenting local online SFA as absent from prior work. TEREL instead contributes a different deep soft-penalty parameterization and an explicit stop-gradient implementation that is easy to compose with standard nonlinear layers. Földiák-style trace learning also exploits temporal persistence [6]; its positive activation trace is not algebraically the same as the signed difference in Eq. (8).

Regularized and blockwise self-supervision. VICReg and Barlow Twins use variance/covariance or cross-correlation penalties to prevent collapse without negatives [2, 3]. Blockwise self-supervised

Table 2: Conceptual comparison. Costs omit learned feedforward parameters.

Method	Temporal objective	Execution	Learned mapping	Auxiliary state
Batch SFA	derivative minimization	batch eigensystem	linear	covariance and whitening
IncSFA	derivative minimization	sample based	linear; deep cascades possible	incremental PCA and MCA state
Bio-SFA	derivative minimization	online neural dynamics	linear network in reported derivation	synaptic/dual variables
Blockwise SSL	paired-view invariance	mini-batch blocks	deep nonlinear blocks	batch statistics
TEREL-local	temporal soft-SFA	sample or minibatch	deep nonlinear layers	$D^2 + 4D + 1$ per layer

learning has already shown that VICReg, Barlow Twins, and contrastive losses can train network blocks without end-to-end backpropagation [7]. TEREL is therefore not distinguished by “a local VICReg loss” alone. Its narrower focus is temporal positives, detached streaming moments, and a same-layer covariance-gradient proxy that can execute one observation at a time.

4 Experimental protocol

The experiments answer three questions. First, after correcting the deep optimizer path, does every declared layer change and does the last layer improve over the same random frozen encoder? Second, on a natural unlabeled stream, does chronological pretraining outperform an identical shuffled learner? Third, does the detached rule execute one sample at a time without a temporal graph while retaining non-collapsed features?

4.1 Data roles and leakage control

MNIST. The official 60,000-example training split is stratified with seed 1701 into 50,000 training and 10,000 validation examples. The official 10,000-example test split is locked until configuration selection is complete. Within each encoder epoch, training examples are randomly arranged into same-class chunks of length 16; all chunk starts are temporal boundaries. Labels are thus used to create temporal supervision. They are removed from the loss batches, but the setting is not self-supervised and we do not describe it as such.

PAMAP2. PAMAP2 contains 100-Hz wearable measurements and activity annotations from nine subjects [8]. We use its 12 protocol activities, discard transition label 0 as instructed by the dataset documentation, and downsample by a fixed stride of 10. Missing sensors are imputed with training-subject means and standardized using training-subject statistics. Subjects 1–6 train the encoder and probe, subject 7 is validation, and subject 8 is the untouched test subject. Subject 9 is excluded before model selection: its protocol recording contains only rope jumping and cannot support a 12-class test. Boundaries are inserted at subject starts, removed-label gaps, and timestamp discontinuities. The encoder iterator returns only features and boundaries; activity labels exist only in the downstream probe and supervised references.

4.2 Models, baselines, and readouts

The primary MLP is input $\rightarrow 512 \rightarrow 256$, with LeakyReLU (negative slope 0.01) and bias. Every frozen representation uses the same linear last-layer probe. A concatenated all-layer probe, when

reported, is secondary and is applied to every compatible deep method. Random uses the identical seed-specific initialization without encoder updates. Supervised BP uses the same hidden dimensions and a linear output. Local supervised contrastive learning uses the same per-layer detachments but receives labels directly. Batch linear SFA and IncSFA are objective-level classical baselines on PAMAP2. Direct covariance, shuffled order, undetached time, and shifted lateral input are mechanism controls.

Encoder selection uses three selection seeds (101, 202, 303). The first four preregistered ReLU configurations all failed the unchanged variance-fidelity criterion on MNIST. We retained that round, fixed LeakyReLU with slope 0.01, and repeated the same four configurations before test access; the cumulative eight remain below the protocol ceiling of twelve. The grid varies AdamW learning rate, λ_V , and λ_C while fixing $\lambda_S = 1$, statistics momentum 0.9, and lateral momentum 0.99. The state rates were fixed by validation-only collapse pilots before the sweep. All representation methods share a 30-epoch AdamW probe with the same batch size and learning rate. The common encoder budget is 10 epochs; exceptions such as closed-form batch SFA report their own sample/pass counts and do not support efficiency claims.

4.3 Confirmatory analysis and test gate

Central results use seeds 1001–1005. We report each raw seed, the mean, sample standard deviation, and a percentile 95% bootstrap interval of the mean from 10,000 resamples (analysis seed 260803). Primary contrasts pair methods by seed and bootstrap the five paired differences. MNIST’s primary metric is accuracy; PAMAP2’s is macro-F1 over all 12 protocol activities. Accuracy and balanced accuracy are secondary.

Before any held-out model evaluation, all implementation and split tests must pass, the validation ledger and selected configurations must be complete, the protocol and code hashes must match a frozen manifest, the tracked worktree must be clean, and the command must include an explicit test flag. The runner enforces these conditions. Test results may narrow claims but cannot trigger new tuning.

5 Results

Confirmatory result tables are generated only after the held-out manifest gate. This draft intentionally contains no inherited numerical values from the rejected implementation.

5.1 Corrected depth fidelity

Every confirmatory run records the ℓ_2 parameter change for every declared encoder layer and the change in every lateral state. A run is invalid if any intended layer is unchanged. This directly guards against the optimizer registration defect that invalidated the submitted deep results. The BP model is constructed from the same declarative dimension list, and a regression test executes each hidden layer, including the historical three-layer comparison.

5.2 What MNIST can and cannot establish

The MNIST experiment asks whether the corrected local rule can exploit a controlled persistent ordering and produce a useful final representation. Its paired comparison to the identical random encoder isolates learning beyond random features; local supervised contrastive learning tests whether explicit same-class attraction is a stronger use of the labels already present in the ordering. Even a favorable result would establish label-assisted local representation learning, not learning from an unlabeled stream.

5.3 Natural temporal order on PAMAP2

The decisive self-supervised contrast holds architecture, initialization, optimizer, sample count, probe, and labels fixed while permuting only the encoder stream. Chronological minus shuffled macro-F1 therefore estimates the downstream value of natural adjacency for this learner. Batch SFA and IncSFA place the effect relative to classical slowness algorithms; random and BP mark untrained and fully supervised references.

5.4 Streaming execution and cost

The batch-size-one run uses detached previous activations and population state, checks that stored tensors have no autograd parent, and verifies that dynamic state is constant as stream length grows. We report its feature variance, throughput, peak allocated device memory, parameter bytes, optimizer bytes, and the $D^2 + 4D + 1$ dynamic-state bytes separately. Minibatch execution is a speed comparison, not evidence that the objective itself requires a batch.

6 Limitations

TEREL inherits the temporal-coherence assumption: rapidly changing or aliased latent causes can make adjacent observations misleading. The MNIST ordering is synthetic and label-derived; PAMAP2 is a modest sensor benchmark, not evidence of ImageNet-scale learning. Subject-disjoint activity recognition also contains distribution shift, missing activities for some subjects, and class imbalance.

The dense lateral operator is the largest conceptual cost. It gives bounded history but $O(D^2)$ state and within-layer communication. Sparse, low-rank, or learned approximations are future work and should be evaluated rather than assumed equivalent. The anti-collapse term can fail if its running state responds too slowly. Our fixed state rate was selected by explicit collapse diagnostics, and the first complete ReLU validation round was rejected by the unchanged variance criterion before LeakyReLU was fixed for the second round.

Our locality claim concerns credit assignment, not a complete biological model. Standard optimizers, exact activation derivatives, synchronized matrix operations, and dense state remain. Likewise, reporting memory and throughput does not establish energy efficiency on neuromorphic hardware.

Finally, the earlier exploratory recurrent experiment mixed TEREL losses with prediction gradients in a fusion module, used one seed, and underperformed its BPTT control. It is not part of the revised empirical claim. A recurrent extension will require an exact separation of prediction and representation paths, repeated seeds, and matched random-state controls.

7 Conclusion

Temporal coherence, variance preservation, and decorrelation form a compelling representation-learning recipe because they connect modern regularized self-supervision to SFA. TEREL turns that recipe into a concrete deep local rule: each nonlinear layer receives a detached temporal error, a gated neuron-local expansion signal, and a tracked same-layer decorrelation signal. The resulting credit assignment is local across depth and, when the preceding activation is detached, across time. Its history is bounded, although its lateral state is quadratic in width.

The corrected experiments are designed to keep this intuition intact while drawing the claim boundary sharply. Label ordering is treated as supervision; natural-stream evidence is evaluated separately;

every layer update and split role is tested; and held-out results are paired, repeated, and provenance locked. The value of TEREL is therefore not a claim to have invented SFA’s objective, but a testable deep parameterization for learning useful temporal representations without end-to-end or through-time credit assignment.

References

- [1] Laurenz Wiskott and Terrence J Sejnowski. Slow feature analysis: Unsupervised learning of invariances. *Neural computation*, 14(4):715–770, 2002.
- [2] Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning. *arXiv preprint arXiv:2105.04906*, 2022.
- [3] Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stéphane Deny. Barlow Twins: Self-Supervised Learning via Redundancy Reduction. *arXiv preprint arXiv:2103.03230*, 2021.
- [4] Varun Raj Kompella, Matthew Luciw, and Jürgen Schmidhuber. Incremental Slow Feature Analysis: Adaptive and Episodic Learning from High-Dimensional Input Streams. *Neural Computation*, 24(11):2994–3024, 2012.
- [5] David Lipshutz, Charles Windolf, Siavash Golkar, and Dmitri Chklovskii. A Biologically Plausible Neural Network for Slow Feature Analysis. In *Advances in Neural Information Processing Systems*, volume 33, pages 14986–14996, 2020.
- [6] P Földiák. Learning invariance from transformation sequences. *Neural computation*, 3(2):194–200, 1991.
- [7] Shoaib Ahmed Siddiqui, David Krueger, Yann LeCun, and Stéphane Deny. Blockwise Self-Supervised Learning at Scale. *Transactions on Machine Learning Research*, 2024.
- [8] Attila Reiss. PAMAP2 Physical Activity Monitoring, 2012.

A Gradient derivations

For valid detached pairs,

$$\frac{\partial}{\partial z_t} \frac{1}{N_I D} \sum_s I_s \|z_s - \text{sg}(z_{s-1})\|^2 = \frac{2I_t}{N_I D} (z_t - p_t). \quad (14)$$

Because g , m , A , and q_t are detached,

$$\frac{\partial L_V}{\partial z_t} = -\frac{2}{BD} g \odot (z_t - m), \quad (15)$$

$$\frac{\partial L_C}{\partial z_t} = \frac{1}{BD} A q_t. \quad (16)$$

Combining terms gives Eq. (8), and applying the chain rule to $z_t = \phi(Wx_t + c)$ gives Eq. (9). For undetached adjacent pairs, z_t appears as the current member of pair t and the previous member of pair $t + 1$; summing those derivatives gives Eq. (10). Finite-fixture regression tests check both signs and the presence of the future term.

For the direct covariance control, let Z_c stack centered rows and $C = \text{offdiag}(Z_c^\top Z_c / B)$. Since C is symmetric,

$$\frac{\partial}{\partial Z_c} \frac{\|C\|_F^2}{D} = \frac{4}{BD} Z_c C, \quad (17)$$

equivalently $4C(z_t - m)/(BD)$ in column-vector notation. The proxy replaces $4C(z_t - m)$ by the detached direction Aq_t and absorbs the positive scale into λ_C .

B Implementation-fidelity tests

The public suite checks:

- the negative variance gradient sign, variance gate, temporal detachment, undetached future derivative, lateral-proxy error bound, and direct covariance fixture;
- nonzero updates for every declared encoder layer and lateral state, with no gradient from a later layer to an earlier layer;
- fixed dynamic-state size and absence of retained autograd graphs;
- exact MNIST split roles, PAMAP2 subject roles, train-only normalization, temporal boundaries, and label-free natural-stream batches;
- execution of every declared BP layer and the corrected legacy version-2 dispatcher;
- batch SFA constraints, boundary-aware derivatives, and numerical validation of the official-algorithm IncSFA port; and
- refusal of held-out evaluation after protocol, ledger, commit, or tracked-worktree drift.

C Reproducibility record

The repository pins Python 3.12 and all direct dependencies in `pyproject.toml` and `uv.lock`. The protocol, selection plan, validation ledger, confirmatory manifest, per-seed JSON records, raw confusion matrices, hardware/software record, and generated paper tables are checksum linked. The core commands are:

```
uv sync --extra test
uv run pytest -q
uv run python -m terel.resubmission.selection \
  --plan configs/resubmission/selection-plan.yaml ...
uv run python -m terel.resubmission.confirmatory run \
  --manifest <manifest.json> --protocol <protocol.md> \
  --validation-ledger <validation-ledger.json> \
  --repository . --allow-test --output <directory> --device cuda
```

Absolute data and artifact roots in the executed plan are retained for provenance; the README gives portable equivalents. Dataset files are not redistributed.

D Complete configurations and raw results

Confirmatory raw-seed tables are generated from the frozen result manifest.